Web Dev Mastery

📘 CSS `display` Property

✅ What We Will Cover

- What is `display` in CSS?
- Difference: `display: none` vs `visibility: hidden`
- Important `display` values:
  - `block`
  - `inline`
  - `inline-block`
  - `none`
  - `contents`
  - `list-item`
  - `table`, `table-row`, `table-cell`
- Bonus: `visibility: hidden`
- When to use which value
- Code examples: HTML + CSS

- Summary table

🔹 What is `display`?

The `display` property determines **how an element is rendered on the page** — as a block, inline, or other visual format. It controls:

- Whether the element starts on a new line
- How much space it takes
- Whether it respects `width`/`height`

⚠️ `display: none` vs `visibility: hidden`

Property	Behavior
`display: none`	Completely removes element from layout (no space reserved)
`visibility: hidden`	Makes element invisible, but space is still preserved

Example CSS:

```
.hidden {
  display: none;
}

.invisible {
  visibility: hidden;
}
```

Example HTML:

```
<p class="hidden">This will not be visible or take space.</p>
```

```
<p class="invisible">This will be invisible but space is
preserved.</p>
```

📚 Detailed `display` Values (Excluding Flex & Grid)

---

1. `display: block`

- Takes up full width of the container
- Always starts on a new line
- Allows width/height/margin/padding

📌 Default for: `<div>`, `<p>`, `<section>`, `<article>`

✅ **Use When:**

- Creating sections, containers, or layout blocks

💡 **Example:**

html

```
<div class="block-example">I'm block</div>
```

css

```
.block-example {
  display: block;
  width: 300px;
  background: lightblue;
}
```

---

2. `display: inline`

- Takes only the width required by content
- Does **not** start on a new line
- Cannot apply `width`, `height`, `margin-top`, `margin-bottom`

📌 Default for: `<span>`, `<a>`, `<strong>`

✅ **Use When:**

- Wrapping text or styling part of text inline

💡 **Example:**

html

```
<p>This is <span class="inline-example">inline text</span>.</p>
```

css

```
.inline-example {
  display: inline;
  color: red;
}
```

---

3. `display: inline-block`

- Behaves like `inline`, but allows `width`, `height`, and `margin`

✅ **Use When:**

- Styling buttons, badges, or nav links inline while needing size control

💡 **Example:**

html

```
<div class="inline-block-example">Box 1</div>
```

```
<div class="inline-block-example">Box 2</div>
```

css

```
.inline-block-example {
  display: inline-block;
  width: 100px;
  height: 50px;
  background-color: coral;
  margin: 5px;
}
```

---

4. `display: none`

- Removes the element from the layout completely
- No space is reserved
- Useful for toggling visibility with JavaScript

✅ **Use When:**

- Hiding modals, dropdowns, tooltips until needed

💡 **Example:**

html

```
<p class="hide-me">This is hidden.</p>
```

css

```
.hide-me {
  display: none;
}
```

---

5. `display: contents`

- Renders only the element's children, not the container itself

- The parent's box disappears, but children remain

⚠️ **Use with caution:**

- May break accessibility or JavaScript DOM traversal

✅ **Use When:**

- Structuring markup purely for semantic or styling reasons

💡 **Example:**

html

```
<div class="contents-example">
  <p>Item A</p>
  <p>Item B</p>
</div>
```

css

```
.contents-example {
  display: contents;
}
```

---

6. `display: table`, `table-row`, `table-cell`

Mimics traditional HTML table layout.

✅ **Use When:**

- Custom table layouts using `<div>` instead of `<table>`

💡 **Example:**

html

```
<div class="table">
  <div class="table-row">
    <div class="table-cell">Name</div>
    <div class="table-cell">Age</div>
  </div>
```

```
  <div class="table-row">
    <div class="table-cell">Suman</div>
    <div class="table-cell">24</div>
  </div>
</div>
```

css

```
.table {
  display: table;
}
.table-row {
  display: table-row;
}
.table-cell {
  display: table-cell;
  border: 1px solid black;
  padding: 5px;
}
```

---

7. `display: list-item`

- Makes an element behave like a `<li>`
- Adds a bullet or number automatically

✅ **Use When:**

- Customizing list layouts or pseudo-lists

💡 **Example:**

html

```
<div class="list-item">List item 1</div>
```

css

```
.list-item {
  display: list-item;
  list-style-type: disc;
```

```
  margin-left: 20px;
}
```

🎯 Bonus: `visibility: hidden`

- Makes the element invisible but still occupies layout space

✅ **Use When:**

- You want to hide content temporarily **without affecting layout**

💡 **Example:**

html

```
<p class="invisible">You can't see me, but I'm here!</p>
```

css

```
.invisible {
  visibility: hidden;
}
```

📌 Summary Table

Value	Description	Starts New Line	Width/Height Allowed	Space Taken
`block`	Full width container	✅ Yes	✅ Yes	✅ Yes
`inline`	Flows within text	❌ No	❌ No	✅ Yes
`inline-block`	Inline with box model	❌ No	✅ Yes	✅ Yes
`none`	Completely removed from layout	❌ No	❌ No	❌ No
`contents`	Renders children only	❌ Depends	❌ No	✅ Yes
`list-item`	Like a bullet point	✅ Yes	✅ Yes	✅ Yes
`table`	Behaves like `<table>`	✅ Yes	✅ Yes	✅ Yes

`table-row`	Behaves like `<tr>`	✅ Yes	✅ Yes	✅ Yes
`table-cell`	Behaves like `<td>`	❌ No	✅ Yes	✅ Yes
`visibility: hidden`	Hides visually, keeps layout	❌ No	✅ Yes	✅ Yes

✅ When to Use What?

Use Case	Recommended Display
Layout container or section	`block`
Inline styling for short content	`inline`
Buttons or tags inside a line	`inline-block`
Hide an element completely	`display: none`
Hide an element but keep layout	`visibility: hidden`
Custom table structure	`table`, `table-cell`, `table-row`
Custom list appearance	`list-item`
Keep children but not container	`contents` (careful use)